When Does Shared-Bus Arbitration Fail to Protect Streaming Sensors During TinyML Inference?

A SystemC Failure-Map Study on a RISC-V Virtual Prototype

Abdelrahman Youssef
Computer Engineering Department
King Fahd University of Petroleum and Minerals (KFUPM), Saudi Arabia
COE 407/661 System on Chip — term paper

***Abstract*—A TinyML system-on-chip shares one interconnect between three kinds of traffic: a CPU that runs a hard real-time control loop, a streaming sensor whose small FIFO is drained by DMA, and a DNN accelerator whose DMA moves weights and activations. This paper builds a request/grant bus arbiter in SystemC/TLM-2.0 inside the RISC-V VP++ virtual prototype, with a STONNE-timed accelerator running the MLPerf Tiny DS-CNN network, and maps where five arbitration policy variants—round-robin, two fixed priorities, a buffer-status priority and a budget-with-fragmentation regulator—stop meeting three limits fixed in advance from datasheets and the real-time literature: no sensor overflow within a FIFO of 8 (strict) or 1280 (relaxed) samples, no missed control-loop deadline, and at most 10 % inference delay. A sufficient FIFO bound is derived and holds at all 100 points where it applies (measured minimum 0.50–1.00 of the bound, 229 vs. 230 samples at the tightest point). Over 240 configurations and 1,036 simulations, the four non-throttling policies give minimum FIFO depths within one sample of each other: on a bus whose bursts cannot be interrupted, the sensor always waits for one in-flight DNN burst whatever the grant order. The outcome is decided instead by the DNN burst length (1024-byte bursts make the control loop miss 17 of 80 deadlines under every policy; 256-byte bursts meet it at 8 % inference cost), by the sensor's drain threshold, and by the 10-cycle setup of every burst. All limits are met with an 8-sample FIFO up to 3.2 MB/s, only with a ≈145-sample FIFO at 32 MB/s, and not at 100 MB/s, where the sensor's bus share alone exceeds the delay margin. Under this bus model, no feasible point exists where every established policy fails, which locates a possible research target in settings where grant order does matter.**

***Index Terms*—bus arbitration, DMA, streaming sensor, FIFO sizing, real-time, TinyML, DNN accelerator, SystemC, TLM-2.0, RISC-V virtual prototype**

I. Introduction

Microcontroller-class SoCs now run DNN inference next to the sensor that feeds them: keyword spotting next to a microphone, vision next to a camera interface, and, at the high end, signal classification next to an RF analog-to-digital converter. In such a chip the accelerator's DMA, the sensor's DMA and the CPU share one interconnect. A sensor that cannot get the bus in time overflows its FIFO and loses samples; a CPU control loop that waits too long for the bus misses its deadline; and every cycle given to the sensor or the CPU delays the inference. The designer chooses three things that decide this trade-off: the arbitration policy, the DMA burst length, and the FIFO size with its drain threshold.

This paper measures, in simulation, where standard arbitration policies stop protecting a streaming sensor and a CPU deadline while a DNN runs, and what that protection costs in inference time. It makes four technical contributions:

- a request/grant shared-bus model in SystemC/TLM-2.0 with three masters, five policy variants and a handover cost on ownership change, integrated into the RISC-V VP++ virtual prototype with a STONNE-timed DNN accelerator (Section III);
- a sufficient FIFO-depth bound for a sensor whose DMA drains θ samples per burst, with its proof, used as a one-sided check of the simulator (Section IV);
- acceptance limits fixed before any sweep from datasheets and the real-time literature (Section V);
- a failure map over policy, DNN burst length, sensor rate, drain threshold and FIFO depth, with the exact minimum FIFO depth found by bisection, and a test of whether any feasible point defeats every policy (Section VII).

The study is a case study under an explicitly stated bus model, not a hardware timing guarantee. All code, scripts and the 1,036 result files are public.[1]

II. Background and Related Work

Accelerator contention on shared buses. Restuccia et al. restore fairness in AXI round-robin arbiters [1], model bus contention of FPGA accelerators analytically [2], bound memory access times with several accelerators [3] and make DNN acceleration time-predictable [4]. Brilli et al. measure CPU/accelerator memory interference on FPGA SoCs [5], and Oliveira et al. study shared-resource contention and timeliness on low-end MCUs [6]. Pujol et al. discuss the limits of AXI4 time predictability [7].

Arbitration and regulation. Real-time interconnects include AXI-IC^{RT} [8], buffer-aware arbitration [9] and bandwidth-control arbitration evaluated in SystemC [10]. AXI-REALM regulates traffic per manager with budgets and burst fragmentation [11]; Carfield uses a similar traffic shaper in silicon [12], and Benz's thesis covers real-time DMA and sensor I/O [13]. Deadline-aware schedulers prioritise accelerator traffic by progress [14], [15] or arbitrate a shared last-level

[1] https://github.com/AbdelRahmanAtef1911/soc-accel-project

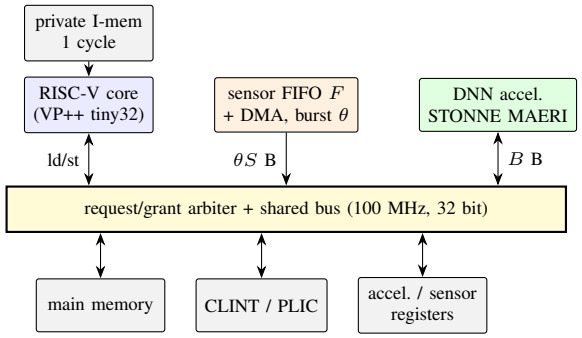


Fig. 1. Modelled SoC. The CPU fetches instructions from a private memory; its loads and stores, the sensor DMA and the DNN DMA compete for one shared bus.

cache [16]; on phones, NPU traffic priority hurts foreground work [17].

DNN timing under contention. Execution time and WCET of CNNs under memory contention are predicted or bounded for FPGA DPUs [18]–[20]; multiple DNNs are scheduled on MCUs with external memory [21]; contention-aware co-scheduling [22] and data-movement-aware accelerator scheduling [23] reduce interference; a compile-time schedule of shared-memory accesses gives a time-triggered alternative [24].

Modelling. STONNE gives cycle-level DNN accelerator timing [25]; the RISC-V VP and VP++ are SystemC virtual prototypes [26], [27]; loosely-timed SystemC DNN models with memory contention are shown in [28]. The workload is DS-CNN from MLPerf Tiny [29]; DORY [30] and μNPU benchmarking [31] show that data movement matters on MCU-class accelerators.

This paper uses the mechanisms above as baselines—round-robin, fixed priority, a buffer-status priority inspired by [9], and a budget with fragmentation inspired by [11], [12]—and asks where, together, they meet a streaming sensor's FIFO limit, a hard CPU deadline and an inference-delay margin. The two derived variants keep only the core mechanism of the cited designs; AXI-REALM, for example, also has monitoring and further regulation that are not modelled.

III. System and Bus Model

Platform. The platform (Fig. 1) is the RISC-V VP++ *tiny32* core at 100 MHz [27] with a memory-mapped DNN accelerator modelled in SystemC/TLM-2.0, whose compute time comes from STONNE (MAERI, 256 multipliers, distribution bandwidth 256) [25]. The accelerator runs convolutions natively, tiles the layer (4 tiles) and double-buffers DMA and compute; it signals completion by interrupt and the CPU sleeps (WFI) meanwhile. CPU instruction fetches come from a private 1-cycle memory (an instruction TCM); the CPU's loads and stores—stack, filter state, peripheral registers—use the shared bus. This choice is stated because it matters: with fetches on the bus the CPU contends on every instruction, and with a data TCM it would barely contend at all.

Bus. One 32-bit bus at 100 MHz connects all masters to one memory. A request/grant arbiter grants the bus only when it is free; each master has at most one outstanding request; a granted burst cannot be interrupted (no split transactions). A burst of b bytes holds the bus for

$$c(b) = 10 + \lceil b/4 \rceil \text{ cycles}, \tag{1}$$

a CPU access for one cycle. When ownership changes, the next tenure starts $a = 1$ cycle later (handover); consecutive grants to the same master pay nothing. Requests made in the same simulated instant are collected for one delta cycle before the arbiter decides, so the policy, not the SystemC process order, resolves ties. The CPU port synchronises the core's local time before every access and performs the access only after the grant, so, e.g., an accelerator START write takes effect inside the CPU's tenure.

Masters. (i) The CPU runs a control task released every 62 μs by the CLINT timer (16-tap FIR; deadline = next release) and sleeps in between. (ii) The DNN DMA moves bursts of $B \in \{64, 256, 1024\}$ bytes. (iii) The sensor delivers one $S = 4$-byte sample every $1/R$ into a FIFO of F samples; a sample arriving at a full FIFO is dropped. Its DMA requests the bus when the FIFO holds θ samples, moves exactly θ samples per burst, and frees them when the burst completes (a conservative removal rule). No rule reads F, so as long as nothing overflows, request times do not depend on F.

Policy variants. (1) round-robin; (2) sensor-first fixed priority (sensor>CPU>DNN); (3) CPU-first (CPU>sensor>DNN); (4) buffer-status: the sensor competes round-robin until its FIFO holds 2θ samples, then has top priority (simplified, after [9]); (5) budget with fragmentation: DNN transfers are split into 64-byte fragments, and the DNN may be granted at most X bytes per aligned window of 1000 cycles, $X = \rho \cdot 4000$ with $\rho \in \{0.25, 0.5\}$; eligible requests are served round-robin (simplified, after [11], [12]).

Workload and measurement. After a warm-up of 16 control periods, one DS-CNN inference (10 layers, timing-only tensors, no feature extraction) is released; T_{acc} runs from that release to the end of the last layer, and its deadline is $T_{\text{inf}} = 5$ ms. Overflows, control-task misses and bus occupancy are counted in the window $[\text{release}, \text{release}+T_{\text{inf}})$. MLPerf Tiny supplies only the network; the sensor, CPU and bus scenario is this paper's model, so the results are not MLPerf Tiny results [32]. The DNN delay ΔT_{acc} is measured against the no-sensor run at the same B without throttling, so the throttling of variant 5 counts as a protection cost.

IV. A Sufficient FIFO Bound

Under sensor-first priority no master outranks the sensor, so after requesting it waits at most for one lower-priority tenure already in progress plus the handover. With blocking $\beta = c(B)$ cycles and its own burst $\sigma = c(\theta S)$, the worst time from request to burst completion is $L = \beta + a + \sigma$ cycles. A periodic source delivers at most $\lceil RL/f \rceil$ samples in any half-open interval of L/f seconds.

TABLE I
ACCEPTANCE LIMITS, FIXED BEFORE THE SWEEPS

Limit	Value
Sensor data	zero FIFO overflows in the window
FIFO size	strict $F \leq 8$ samples; relaxed $F \leq 1280$ samples
CPU	zero late or lost releases of the 62 μs task
DNN deadline	$T_{\mathrm{acc}} \leq T_{\mathrm{inf}} = 5$ ms
DNN delay	$\Delta T_{\mathrm{acc}} \leq 10\,\%$ (also 5 % and 25 %)

TABLE II
NO-SENSOR RUNS (IDENTICAL FOR VARIANTS 1–4)

DNN setting	T_{acc} (cycles)	control task late/lost	worst resp. (cyc)
$B = 64$	205,163	0/80	1,207
$B = 256$	158,738	0/80	3,212
$B = 1024$	146,389	17/80	10,134
budget, $\rho = 0.5$	234,179	0/80	1,182
budget, $\rho = 0.25$	434,070	0/80	722

Proposition. No sample overflows if

$$\text{(C1)} \quad F \geq \theta + \lceil RL/f \rceil, \tag{2}$$

$$\text{(C2)} \quad \lceil RL/f \rceil \leq \theta. \tag{3}$$

Proof. By induction over drain rounds. A request is raised when occupancy reaches θ. Until its burst completes at most $\lceil RL/f \rceil$ samples arrive, so the peak occupancy is $\theta + \lceil RL/f \rceil \leq F$ by (C1) and no arriving sample finds the FIFO full. At completion θ samples leave, so at most $\lceil RL/f \rceil \leq \theta$ remain by (C2); the next request is again raised at occupancy θ, and the argument repeats. □

For round-robin, β becomes one tenure of each other master plus a handover, $\beta_{RR} = c(B) + 1 + a$. No bound is claimed for CPU-first, whose CPU demand the model does not bound. The bound is sufficient, not tight, and is used one-sidedly: every simulated point that satisfies (C1)–(C2) must show zero overflows.

Bus-overhead estimate. Because the double-buffered DS-CNN is largely bus-bound, a sensor that takes a bus share $s = (R/\theta)(c(\theta S) + a)/f$ delays it by roughly $s/(1 - s)$. The smallest share reachable within a FIFO limit uses the largest allowed θ; where even that share exceeds the delay margin, no arbiter can meet the margin, and the point is excluded from the gap test as *bandwidth-infeasible*. This is an estimate, not a bound, and is checked against the simulations below.

V. ACCEPTANCE LIMITS

The limits were fixed before the sweeps (Table I). The sensor limit follows from the camera-interface note that a FIFO has no overrun protection when the bus cannot keep up [33]. The strict FIFO size is typical of on-chip peripheral FIFOs: an 8-word audio FIFO [34], an 8-sample ADC FIFO [35] and a 4-word camera FIFO [33]; the relaxed size is the 5120-byte camera-pipeline FIFO reported for a recent MCU [36] (a forum report). The control task is hard real-time [37]. The 10 % delay margin has no standard value in the literature; it is a stated design choice, and the gap test is repeated at 5 % and 25 %.

VI. VALIDATION

Unit tests. Seven tests drive the arbiter with stub masters. Each variant grants three simultaneous requests in its documented order. Under sensor-first priority with 1024-byte DNN bursts ($c = 266$ cycles), the sensor's worst wait is 267 cycles $= \beta + a$, and under round-robin 268 cycles against a bound of 270. With three saturating masters and equal 64-byte requests, round-robin grants 2470/2469/2469 times; with unequal tenures it still equalises grants, and each master's busy share equals its tenure divided by the sum of tenures plus handovers. Fixed priority starves the lower masters under saturation. The budget variant grants exactly 960 of 1000 allowed bytes in every aligned window. In 13,794 alternating tenures, the 13,793 handovers equal the ownership changes, and no tenures overlap.

Regression. With handover $a = 0$ the request/grant bus reproduces the earlier accelerator testbench cycle counts exactly; with $a = 1$ every job is exactly one cycle longer—the one ownership change after the CPU's START write. A CPU-only benchmark run (48 layers in software) is identical to the earlier result, and the earlier bus model is unchanged.

Bound and monotonicity. Over all sweeps, the 50 sensor-first and 50 round-robin points that satisfy (C2) all meet $F_{\min} \leq F_{\mathrm{suf}}$, with $F_{\min}/F_{\mathrm{suf}}$ between 0.50 and 1.00. At 100 MB/s, $B = 1024$, $\theta = 128$, the bound gives 230 samples; the measured minimum is 229 and the sensor's worst wait is 267 cycles. Runs at $F_{\min}$ and at $F = 512$ give identical window results at all six sampled points, confirming that overflow-freedom is monotonic in F, so $F_{\min}$ can be found by bisection.

VII. EXPERIMENTS AND RESULTS

Sweeps. Stage A (124 runs) covered variants 1–4, $B \in \{64, 1024\}$, five sensor rates from 32 kB/s to 100 MB/s and three depths, with θ set to the smallest power of two that satisfies (C2). It showed that this choice of θ minimises the FIFO but maximises the per-burst overhead. Stage B (240 configurations, 912 runs) therefore sweeps θ in powers of two up to 128, adds $B = 256$ and variant 5, and finds the exact minimum depth $F_{\min}$ by integer bisection on $[\theta, 512]$; the CPU and DNN results of the passing run hold for every $F \geq F_{\min}$.

The burst length decides the CPU deadline. Table II shows the runs without a sensor. With 1024-byte bursts the control task misses 17 of 80 releases under *every* policy, CPU-first included: each of its ≈100 loads and stores may wait for an in-flight 266-cycle burst, and its worst response (10,134 cycles) exceeds the 6,200-cycle period. 256-byte bursts keep the worst response at 3,212 cycles for 8 % more inference time than 1024-byte bursts; 64-byte bursts cost 40 %. The budget variant protects the CPU at $B = 1024$ but throttles the DNN by 60 % ($\rho = 0.5$) to 197 % ($\rho = 0.25$).

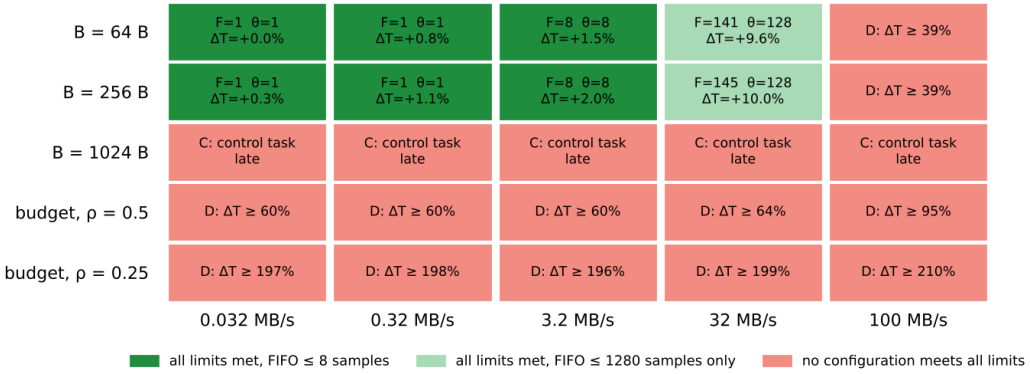


Fig. 2. Failure map (delay margin 10 %). For each DNN setting and sensor rate, the best configuration over variants 1–4 and θ (variants 1–4 behave alike, Table III); text gives its $F_{\min}$, θ and ΔT_{acc}, or the limit that no configuration meets (C: control task, D: DNN delay).

TABLE III
POLICY INVARIANCE OVER 50 (BURST, RATE, θ) POINTS, VARIANTS 1–4

Quantity	largest spread across policies
minimum FIFO depth $F_{\min}$	1 sample (3.3 %)
DNN delay ΔT_{acc}	2.6 percentage points
CPU deadline outcome	differs at 3 of 50 points

Failure map. Fig. 2 summarises the map. All limits are met with the strict 8-sample FIFO up to 3.2 MB/s, with 64- or 256-byte bursts and $\theta = 8$ at 3.2 MB/s (ΔT_{acc} 1.5–2.0 %). At 32 MB/s only the relaxed FIFO works, with $\theta = 128$ and $F_{\min} = 141$–145 at $\Delta T_{\text{acc}} \approx 10\,\%$. At 100 MB/s no configuration meets the margin: even with $\theta = 128$ the sensor needs about 27 % of the bus. 1024-byte bursts fail the control task everywhere, and the budget variants fail the delay margin everywhere.

Grant order hardly matters. Fig. 3 and Table III show the central result: the four non-throttling variants give minimum FIFO depths within one sample of each other at every point. On a bus whose bursts cannot be interrupted, the sensor's worst wait is one in-flight DNN burst whatever the order—the bound's β—and the CPU's one-cycle accesses are the only thing the order changes. Round-robin adds one CPU access to that wait; sensor-first removes it. The DNN delay is likewise set by how much of the bus the others use, not by when they use it, because the arbiter is work-conserving.

The delay follows the sensor's bus share. Fig. 4 compares the smallest reachable delay with the overhead estimate. The estimate tracks the simulations from 3.2 MB/s upward, and no simulated configuration within a FIFO limit meets the delay margin where the estimate says it cannot (0 violations at all three margins and both FIFO limits). With the strict FIFO, the 10-cycle setup of 32-byte sensor bursts already costs about 19 % of the bus at 32 MB/s.

The drain threshold trades FIFO for overhead. Fig. 5 shows the trade-off that θ controls. At 3.2 MB/s, $\theta = 1$ needs a one-sample FIFO but costs about 10 %, while $\theta = 8$ fits the strict limit at 1.5–2.4 %. At 32 MB/s the curve crosses the 10 % margin only near $F \approx 140$; inside the strict limit the best point is $\theta = 4$, $F_{\min} = 7$, at a 43 % delay. Lines of different policies lie on top of each other.

Research-gap test. The plan asked whether a feasible point exists where every established policy fails to meet all limits together. Excluding points that are bandwidth-infeasible by the estimate, the answer is no for both FIFO limits at 5 % and 10 % margins and for the relaxed FIFO at 25 %. One candidate remains: 32 MB/s with the strict FIFO and a 25 % margin. There $\theta = 8$ would meet the margin (ΔT_{acc} 22–25 %) but needs $F_{\min} = 11$–15, because samples keep arriving during the sensor's own 18-cycle burst and handover before any DNN blocking; $\theta = 4$ fits in 7 samples but costs 43 %. No grant order can shorten the sensor's own burst, so this corner is a FIFO-design question—word-by-word removal or ping-pong buffers—not an arbitration one.

VIII. DISCUSSION AND LIMITATIONS

For a designer, the map gives three rules under this bus model: choose the DNN burst length first, from the CPU's worst-case access count and deadline ($B = 256$ here); choose θ next, as the largest value whose bound (C1) fits the FIFO; and expect the remaining DNN delay to be close to the sensor's bus share. Arbitration order is a second-order choice. Regulation by budget protects the CPU but, with the parameters tried, costs more inference time than simply using shorter bursts.

The result is bounded by its assumptions. There is one workload (DS-CNN, double buffered), one sensor and one CPU task, and $T_{\text{inf}} = 5$ ms. The CPU's data share the bus while its instructions do not; a data TCM would remove most CPU failures. Bursts are non-preemptive and there is one outstanding request per master: an AXI-style interconnect with outstanding transactions, split bursts or a preemptive DMA could make grant order matter. Variant 5 was run with one window size and two budgets. θ is restricted to powers of two up to 128, and depths are capped at 512. The 10 % margin is a design choice, which is why the gap test is repeated at 5 % and 25 %.

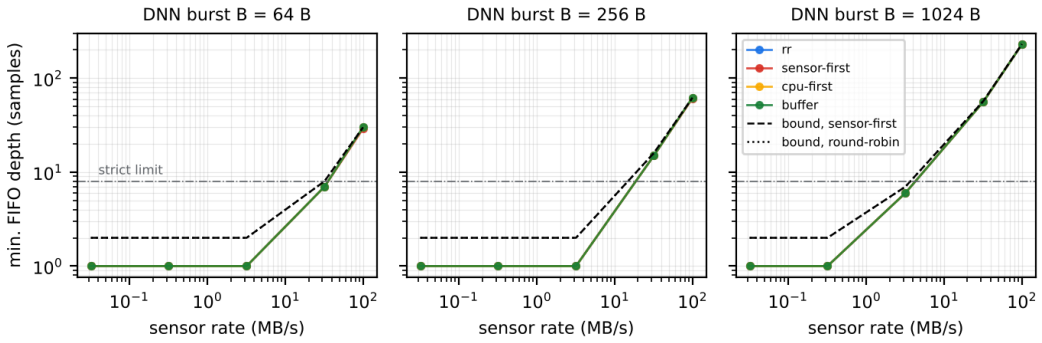


Fig. 3. Exact minimum FIFO depth vs. sensor rate (θ = smallest value meeting (C2)). The four variants overlap; dashed and dotted lines are the sufficient bounds for sensor-first and round-robin.

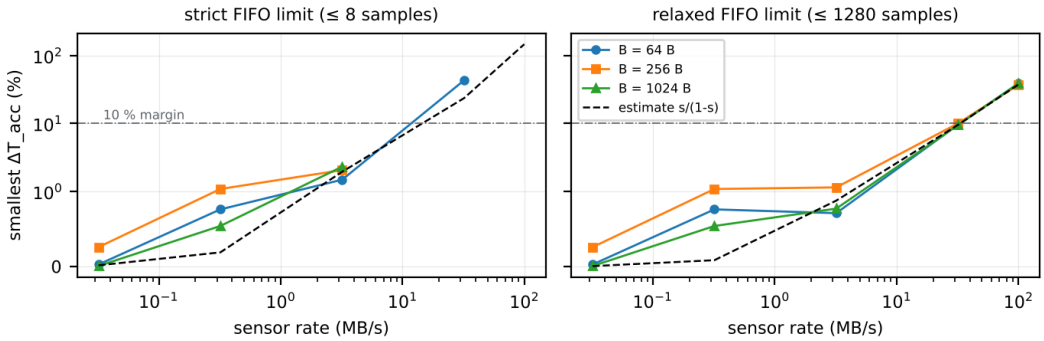


Fig. 4. Smallest DNN delay reachable within each FIFO limit (over variants 1–4 and θ), with the bus-overhead estimate $s/(1-s)$ at the largest θ the limit allows. $B = 1024$ points miss the CPU deadline (Fig. 2).

IX. Conclusion and Future Work

This paper built a validated request/grant bus model in a RISC-V SystemC virtual prototype and mapped where five arbitration policy variants protect a streaming sensor and a hard CPU deadline while a DS-CNN inference runs. A sufficient FIFO bound held at every applicable point and was tight. Under a non-preemptive shared bus with a light CPU, the minimum FIFO and the inference delay are almost independent of the grant order; the DNN burst length, the sensor's drain threshold and the per-burst overhead decide the outcome, and no feasible point defeats every policy. Settings where grant order does matter—heavier CPU traffic, several sensors or DNN masters, outstanding or split transactions, or a preemptive DMA—are the natural next step, and the same platform, bound and scripts can be used to map them.

References

[1] F. Restuccia, M. Pagani, A. Biondi, M. Marinoni, and G. Buttazzo, "Is your bus arbiter really fair? restoring fairness in AXI interconnects for FPGA SoCs," *ACM Transactions on Embedded Computing Systems*, 2019.

[2] ——, "Modeling and analysis of bus contention for hardware accelerators in FPGA SoCs," in *32nd Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 165, 2020, pp. 12:1–12:23.

[3] ——, "Bounding memory access times in multi-accelerator architectures on FPGA SoCs," *IEEE Transactions on Computers*, vol. 72, no. 1, pp. 154–167, 2023.

[4] F. Restuccia and A. Biondi, "Time-predictable acceleration of deep neural networks on FPGA SoC platforms," in *IEEE Real-Time Systems Symposium (RTSS)*, 2021, pp. 441–454.

[5] G. Brilli, A. Capotondi, P. Burgio, and A. Marongiu, "Understanding and mitigating memory interference in FPGA-based HeSoCs," in *Design, Automation & Test in Europe (DATE)*, 2022.

[6] D. Oliveira, W. Chen, S. Pinto, and R. Mancuso, "Shared resource contention in low-end MCUs: A reality check and the quest for timeliness," in *36th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 298, 2024.

[7] Pujol, Abella, Hassan, and Cazorla, "On the time predictability of AXI4," in *ACM Symposium on Applied Computing (SAC)*, 2025.

[8] Z. Jiang, K. Yang, N. Fisher, I. Gray, N. C. Audsley, and Z. Dong, "AXI-IC^{RT}: Towards a real-time AXI-interconnect for highly integrated SoCs," *IEEE Transactions on Computers*, vol. 72, no. 3, pp. 786–799, 2023.

[9] Roh, Jun, Bang, and Chung, "Jitter-conscious bus arbitration scheme for real-time systems," *IEICE Transactions on Fundamentals*, vol. E92.A, no. 2, 2009.

[10] S. Ibarra-Delgado, R. Sandoval-Arechiga, J. R. Gómez-Rodríguez, M. Ortíz-López, and M. Brox, "A bandwidth control arbitration for SoC interconnections performing applications with task dependencies," *Micromachines*, vol. 11, no. 12, p. 1063, 2020.

[11] T. Benz *et al.*, "AXI-REALM: Safe, modular and lightweight traffic monitoring and regulation for heterogeneous mixed-criticality systems," *IEEE Transactions on Computers*, vol. 74, no. 9, 2025.

[12] A. Garofalo *et al.*, "A reliable, time-predictable heterogeneous SoC for AI-enhanced mixed-criticality edge applications," arXiv:2502.18953, 2025.

[13] T. E. Benz, "Development of an energy-efficient and real-time data movement strategy for next-generation heterogeneous mixed-criticality systems," Ph.D. dissertation, ETH Zürich, 2026, arXiv:2602.09554.

[14] H. Usui, L. Subramanian, K. Chang, and O. Mutlu, "SQUASH: Sim-

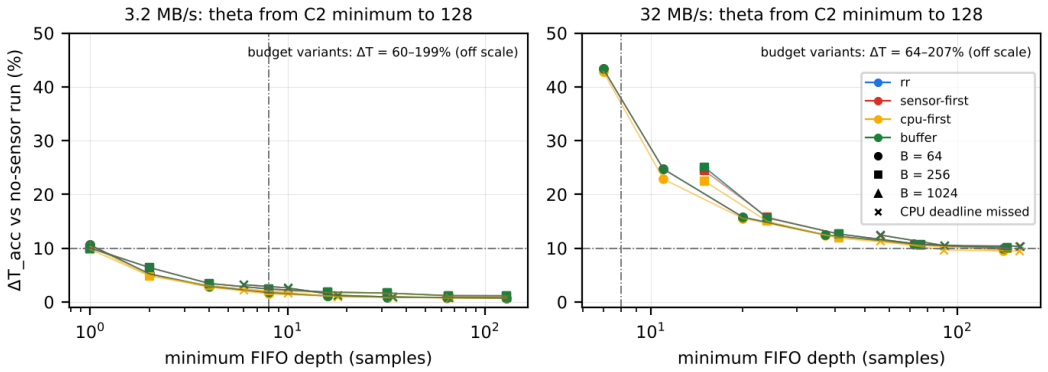


Fig. 5. Trade-off between minimum FIFO depth and DNN delay as θ grows (one line per policy and burst length; $\times$: CPU deadline missed). Dash-dotted lines: strict FIFO limit and 10 % margin.

ple QoS-aware high-performance memory scheduler for heterogeneous systems with hardware accelerators," Carnegie Mellon University, Tech. Rep., 2015, arXiv:1505.07502.

[15] ——, "DASH: Deadline-aware high-performance memory scheduler for heterogeneous systems with hardware accelerators," *ACM Transactions on Architecture and Code Optimization*, 2016.

[16] A. Agarwal, P. Goel, P. Joseph, P. Ghosh, S. Roy, and P. R. Panda, "FLASH: Deadline-aware flexible LLC arbitration and scheduling for hardware accelerators," *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 6, 2025.

[17] T. Xin, X. Shi, M. Dong, and Z. Mi, "Inference in the shadows: Taming memory bandwidth contention in mobile LLM inference with Sereno," in *USENIX OSDI*, 2026.

[18] Zhang, Yu, Jiang, Guan, Zhan, and Ju, "WCET estimation for CNN inference on FPGA SoC with multi-DPU engines," *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 6, 2025.

[19] S. Goel, R. Kedia, R. Sen, and M. Balakrishnan, "EXPRESS: A framework for execution time prediction of concurrent CNNs on Xilinx DPU accelerator," *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 1, p. 11, 2024.

[20] H. Ren, Y. Yu, W. Zhang, Z. Du, Z. Shao, N. Guan, and L. Ju, "RT-DPU: Scheduling real-time tasks with heterogeneous multi-DPU engines," *IEEE Transactions on Computers*, 2026, early access.

[21] S. Kang, S. Lee, H. Koo, H. S. Chwa, and J. Lee, "RT-MDM: Real-time scheduling framework for multi-DNN on MCU using external memory," in *61st Design Automation Conference (DAC)*, 2024, pp. 1–6.

[22] I. Dagli and M. E. Belviranli, "Shared memory-contention-aware concurrent DNN execution for diversely heterogeneous system-on-chips," in *ACM PPoPP*, 2024.

[23] S. Gupta and S. Dwarkadas, "RELIEF: Relieving memory pressure in SoCs via data movement-aware accelerator scheduling," in *IEEE HPCA*, 2024, pp. 1063–1079.

[24] M. Kirschner, K. Dudzik, and J. Becker, "Work-in-progress: Real-time neural network inference on a custom RISC-V multicore vector processor," arXiv:2410.10340, 2024.

[25] F. Muñoz-Martínez, J. L. Abellán, M. E. Acacio, and T. Krishna, "STONNE: Enabling cycle-level microarchitectural simulation for DNN inference accelerators," *IEEE Computer Architecture Letters*, vol. 20, no. 2, 2021.

[26] V. Herdt, D. Große, P. Pieper, and R. Drechsler, "RISC-V based virtual prototype: An extensible and configurable platform for the system-level," *Journal of Systems Architecture*, 2020.

[27] M. Schlägl, C. Hazott, and D. Große, "RISC-V VP++: Next generation open-source virtual prototype," in *Workshop on Open-Source Design Automation (OSDA)*, 2024.

[28] E. M. Arasteh and R. Dömer, "Fast loosely-timed deep neural network models with accurate memory contention," *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 5, 2024.

[29] C. Banbury *et al.*, "MLPerf tiny benchmark," in *NeurIPS Datasets and Benchmarks Track*, 2021.

[30] A. Burrello *et al.*, "DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs," *IEEE Transactions on Computers*, 2021.

[31] J. Millar *et al.*, "Benchmarking ultra-low-power μNPUs," in *ACM MobiCom*, 2025, arXiv:2503.22567.

[32] MLCommons, "MLPerf inference: Tiny," https://mlcommons.org/benchmarks/inference-tiny/.

[33] *AN5020: Introduction to digital camera interface (DCMI) for STM32 MCUs*, STMicroelectronics.

[34] *RM0432 Reference manual, STM32L4+: Serial audio interface (SAI)*, STMicroelectronics.

[35] *RP2040 Datasheet and Pico C SDK, hardware_adc*, Raspberry Pi Ltd.

[36] ST Community, "DCMI priority versus SDMMC (STM32N6)," https://community.st.com/t5/stm32-mcus-products/dcmi-priority-versus-sdmmc-stm32n6/td-p/886980, 2026.

[37] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.